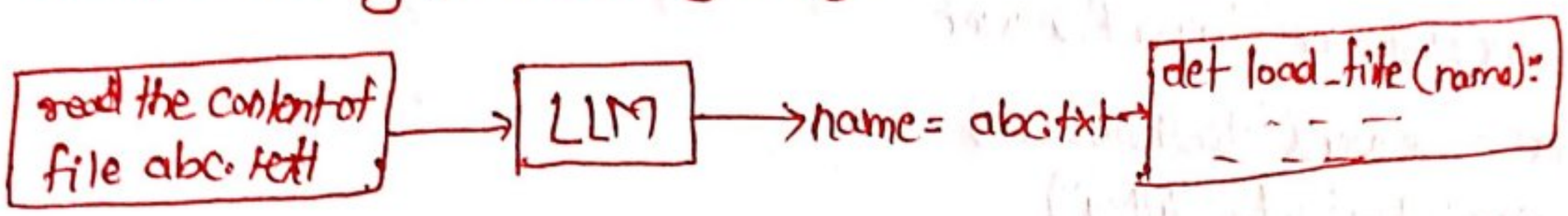
MCP * Model Context Protocol

What is Context

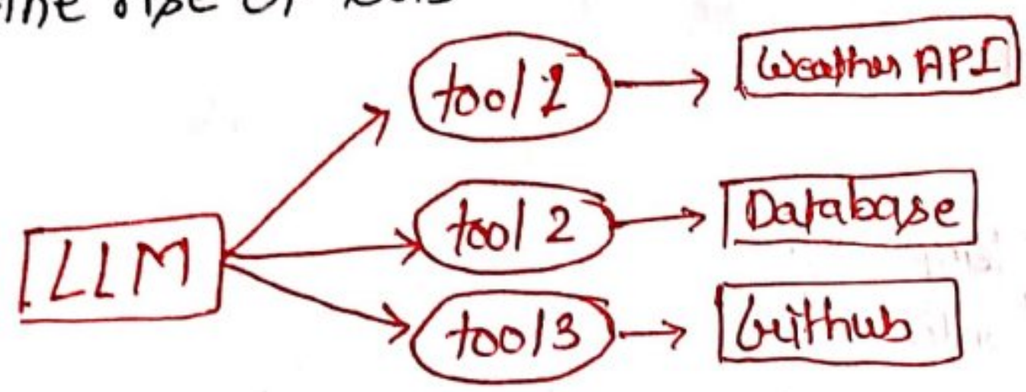
- Context is everything an AI can "see" when it generates a response.
- Context refers to the information (conversation, history) that the LLM used to generate a response.

function calling.

- Function calling is the way using which LLM can call ext functions.



The rise of tools



Implication of tools

- Salesforce integrations for sales teams
- Slack bots that could read msg history and channel context.
- Google Drive connectors for document access and collaboration.
- Database query tools that could analyze company data.
- Github integrations for code review and pull request management.
- Cursor - builds filesystem access and intelligent code search.
- Perplexity - add web browsing and real time information retrieval
- ChatGPT Plus - browsing, file uploads, code ~~edition~~ execute
- Claude - gained computer use capabilities.

Problem with Tools.

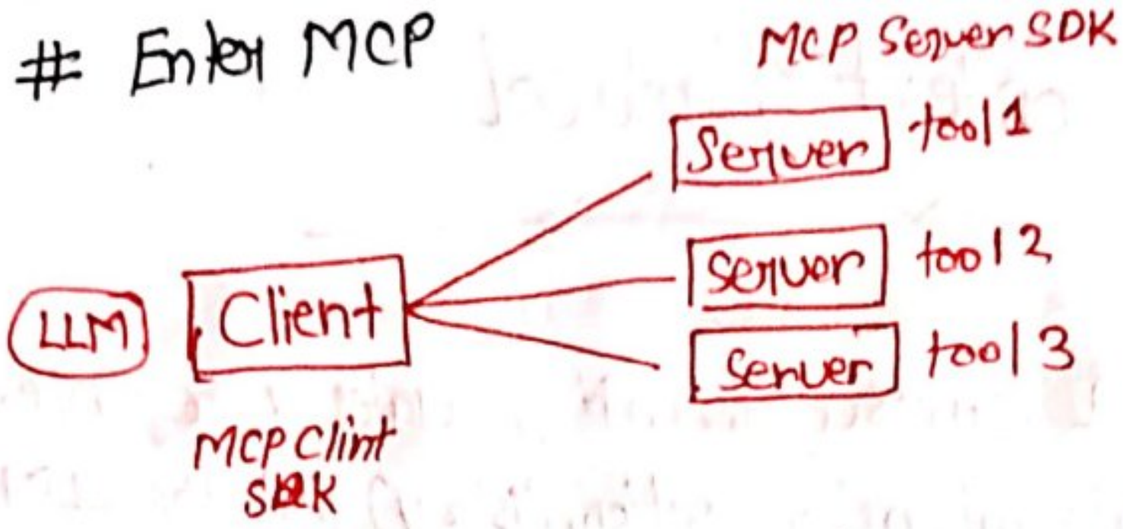
- Integration Problem

  no. of bots x no. of tools

- Maintenance problem
- Security fragmentation
- Cost & time wastage.

Scanned with OKEN Scanner

Enter MCP



MCP code for ~~weather~~ weather

- Client

result = callTool("get_weather", {"city": London"})

- Server

```
from mcp.server. import Server
Server = Server("WetherServer)
@ Server.tool("get_weather")
def get_weather(city: str):
    data = lookup_weather(city)
    return. {
        "temperature": data["temp_c"],
        "Condition": data["Condition"]
    }
Server.start()
```

Server does the Reavy lifting

- authentication with github
- API rate limited
- Data format translation
- Error handling
- Github-specific business logic

The Client has to just connect to the server and say same language

Benefits

- N clients and M servers = M+N integrations
- No maintenance overhead
- reduce cost and time
- Better Security

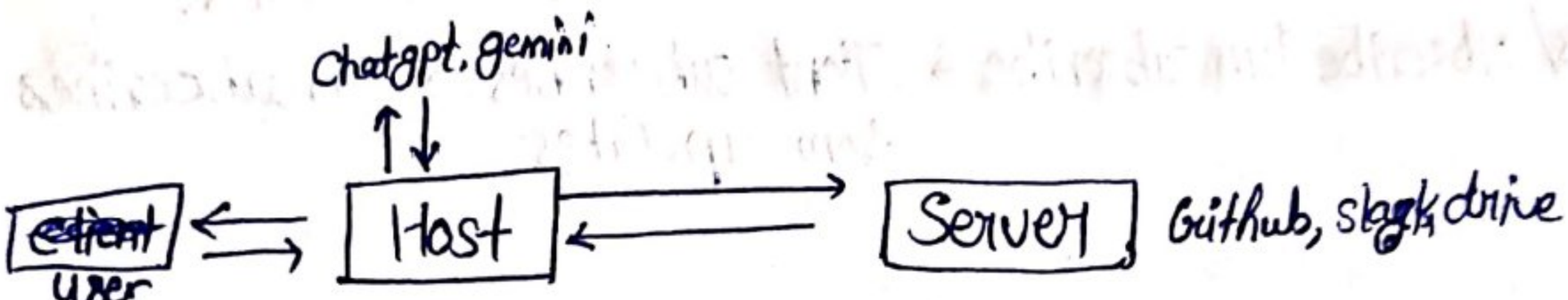
Scanned with OKEN Scanner

MCP Ecosystem

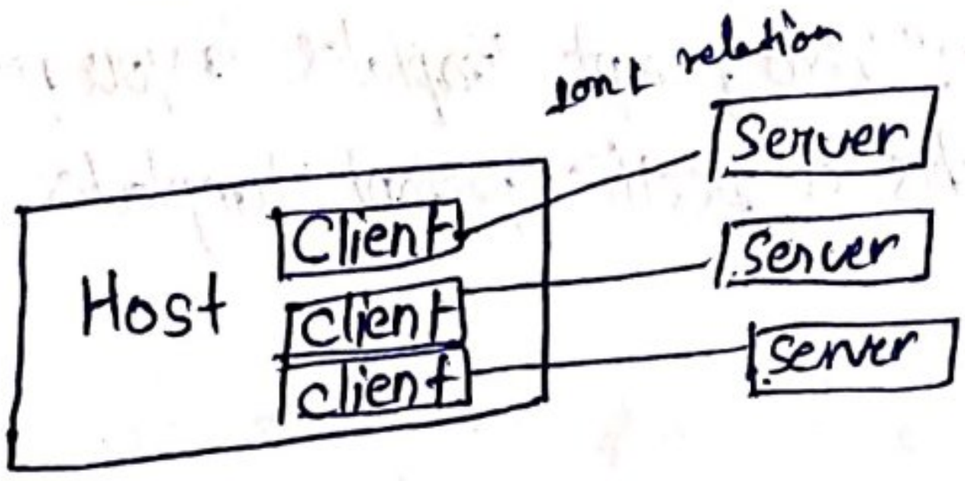
- more AI chatbots supporting MCP → more valuables for services to build MCP servers.
- More MCP server available → More valuables for AI tools to support MCP
- More adoption → more standardization → more ecosystem value
- Not supporting MCP meant being cut off for rapidly growing ecosystem.

MCP Architecture

Simple version



adding client



Benefits

Decoupling
- Safety
- Parallelism

Scalability

Primitives

- Things the server can offer to host
  - tools
  - Resources
  - Prompts.

MCP Primitives

- tools —: Actions the AI ask the server to perform.
- Resources —: Structured data sources that the AI can read.
- Prompts —: Predefined prompt templates or instructions that the server offers to help shape the AI's behavior:

Title: Login bug
Body: The login button does not work. ← normal prompts.

Scanned with OKEN Scanner

Primitives - Standard operations

- Tools
  - tools/list → Client asking the server "what tools do you provide?"
  - tools/call → Client tells the server" Plz run this tool with these arguments."

- Resources
  - resources/list → Client asks" what resources are available?,"
  - resources/read → Client says" Give the content of this resources."
  - resources/subscribe/unsubscribe → Client subscribes or unsubscribes from updates

- Prompts—:
  - prompts/list → Client asks:" what prompt templates do you provide?"
  - prompts/get- Client fetches a specific prompt template.

MCP Data Layer

- Data layer- The data layer is the language and grammer of the MCP ecosystem that everyone agrees upon to communicate
- In MCP, JSON RPC 2.0 serves as the foundation of the data layer.

JSON RPC 2.0

JSON RPC stands for Javascript Object Notation - Remote Procedure Cell

- A RPC allows a program to execute a function on another computer as if it were local, hiding the details of network communication and data transfer. This abstraction masks it easier to build distributed application
- Instead of writting add(2,3) locally, you send a request to the server saying 'please run add with parameters 2 and 3!

Scanned with OKEN Scanner

Request Structure

```
{
 'jsonrpc' : '2.0',
 'method' : 'add',
 'params' : [2,3],
 'id' : 1
}
```

Response Structure

```
{
 'jsonrpc' : '2.0'
 'result' : 5,
 'id' : 1
}
```

Why JSON RPC for Data layer

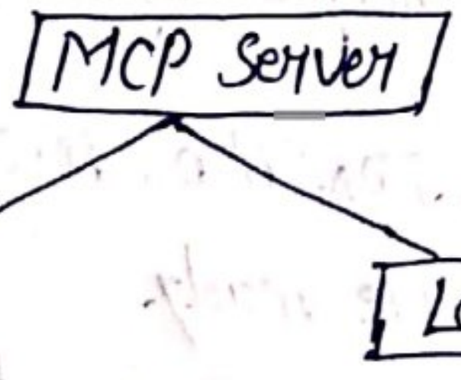
- It's light weight.
- Support by directional communication.
- It is transport-agnostic
- Supports batching.
- Supports notification.

MCP Transport Layer

The Transport Layer is the mechanism that moves JSON-RPC messages between the client and server.

- The choise of transport depends on the type of server.

MCP - Types of server



- a remote server is a prom running on another computer. that you connect to over a network.
- mode of transfer - HTTP/SSE

- a local server is a program running on your own computer.
- mode of transfer — STDIO

Local Server - STDIO

- STDIO refers to the build-in streams every program has.
  stdin (input the program read)
  stdout (output the program write)

Scanned with OKEN Scanner

How does STDIO works

The host launches the server as a subprocess on the same machine.

- The host (client) writes JSON-RPC messages into the server's STDIN.
- The server reads thoes messages, processes them, and writes back responses to it's STDOUT.

Benefits of the STDIO transport

- Fast - data is passed directly b/w processes.
- Secure - no open network port that could be attacked. Communication is only local.
- simple - every language / runtime supports reading/writting from stdin/stdout. No extra libraries required.

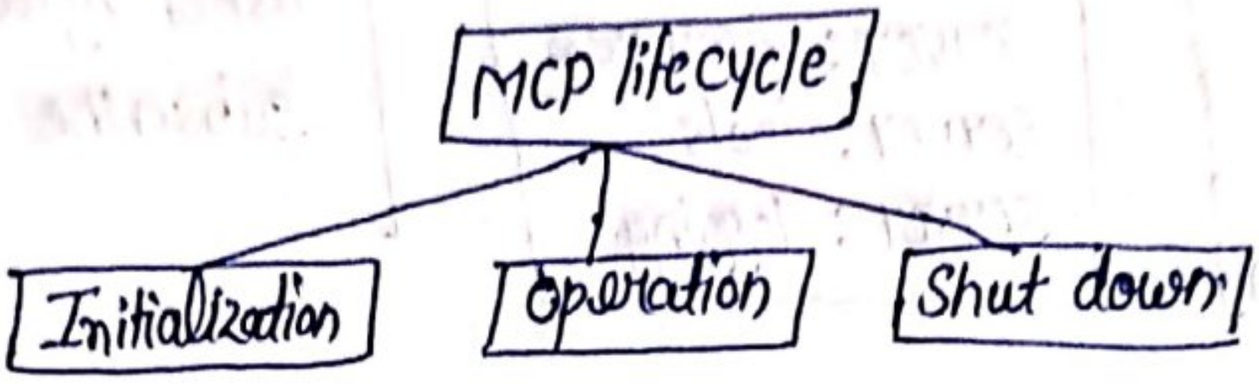
Remote Servers — HTTP + SSE

- Using HTTP allows the host to reach Server running anywhere.
- Host sends JSON RPC requests using POST requests with a JSON payload.
- The transport supports standard HTTP authentication methords (API).
- SSE stands for Server Sent Events and its an extension of HTTP.
- Using SSE the server sends multiple msgs to client over a single open connection.
- Instead of sending one large JSON blob. the server can stream chunks of data as they are ready.
- Ideal for long running tasks or incremental updates.

Scanned with OKEN Scanner

MCP life cycle.

- The MCP Life cycle describes the complete sequence of steps that govern how a host (client) and a server establish, use and end a connection during a session.



Initialization phase - The initialization phase must be the first interaction b/w client and server.

- Establish protocal version compatibility
- Exchange and negotiate capability.

Step 1: The client send a initialize request.
Step 2: The server sends its own capabilities and info.
Step 3: After successful initialization, the client MUST send an initalized notification to indicate it is ready to begin normal operations.

- Now the client and server are connected.

important Rules

- The client should not send requests other than pings before the server has responded to the intialize request.
- The server should not send requests other than pings and logging before receiving the initialized notification.

Scanned with OKEN Scanner

Capability Negotiation

- Client and server capabilities establish which protocol features will be available during the session.

Client — roots Client — sampling Client — elicitation	Server : prompts Server : resources server : tools server : logging	Sub-capabilities listChanged subscribe

Operation Phase

- During the operation phase, the client and server exchange msgs according to the negotiated capabilites.
- Respect the negotiated protocol version.
- Only use capabilites that were successfully negotiated.

Shutdown Phase

- One side (typically the client) initiates shutdown.
- No special JSON RPC shutdown msg is defined.
- Transport layer is responsible for signaling termination.

Shutdown in STDIO

- Client-initiated shutdown (should):
  - Close input stream to the child process (server)
  - Wait for server to exit
  - Send SIGTERM if server does not exit in time.
  - Send SIGKILL if still ~~unle~~ unresponsive.
- Server initiated shutdown
  - Close output stream to the client
  - Exit process

shutdown in HTTP

- Client initiated shutdown (Common case):
  The client (host) closed the HTTP connection (s) it opened to the server..
- Server-initiated shutdown (possible)
  - The server may close the connection from its side.
  - The client must be prepared to detect a dropped connection and handle it (e.g. reconnect if appropriate)

Special Cases

Pings

Pings is a lightweight request/response method defined in MCP.

Purpose-: to check whether the other side (Host or Server) is still alive and connection is responsive.

When is Ping used

- Useful for checking if the otherside is up before full initialized.
- If there's no activity for a while, a client may send periodic pings. Prevents the connection from being dropped silently by the OS, proxies or fire walls.

Error Handling

- Error handling in MCP is how the Host (client) and server signal that something went wrong with a request.
- MCP inherits JSON RPC's standard error object format.

Cause of error

- Unsupported or mismatched protocol version error.
- Calling a method for a capability that wasnot negotiated
- Invalid arguments to a tool
- Internal server failure while processing a request
- Timeout exceeded → client cancels request
- Malformed JSON-RPC msgs.

Scanned with OKEN Scanner

Error object structure

code - integer code that categorizes the error.
message - short human-readable explanation.
data - (optional) extra structured data for debugging or context.

Common Error Codes

Error Code	Name
-32601	Method not found
-32602	Invalid parameters
-32600	Invalid request
-32700	Parse error
-32000 or above	Server defined error

Time Out

Timeout is about ensuring request don't hang forever.

Purpose -
- Protects against unresponsive or overload servers.
- Ensures resources (memory, CPU) aren't held indefinitely
- Gives the user feedback instead of waiting forever.

How timeout works
- SDKs let client sets a per-request timeout (e.g. 30s)
- If the deadline passes with no result → client triggers a timeout.
- Client then sends a cancellation notification to tell the server to stop.
- The server must stop processing that request and not return a result.

Progress Notification
- Purpose: Let the client know a long-running request is still making progress.
- client includes a progress token in the request's _meta.
- server can then send notification / progress update while working.

Scanned with OKEN Scanner

Connectors

- A connector is a built in feature that links claude to MCP server automatically, without the need for manual setup or Configuration
- Most claude Desktop users are non technical end-users who just want Claude to talk to their apps (Notion, Google drive, Github)
- They do not want to run servers, edit JSON or worry about transports.
- The connector system wraps an MCP server behind the scenes and handles authentication via OAuth (Sign in with google, GitHub)
- This keeps things easy, safe and consistent.
- Think of connectors as the 'Appstore' for MCP server - user-friendly click-based, pre curated.

Why not use Connectors always?

- Connectors Are curated & manged.
  - Connector are officially built, hosted and maintained by Anthropic
  - They come with oauth login flows, managed security, rate limits and guaranteed stability.
- MCP is an open standard
  - MCP is designed so anyone can write a server.
  - Forcing everthing through Connectors would close the ecosystem and make you dependent on Anthropic to approve or publish server.

Scanned with OKEN Scanner

Fast MCP or MCP SDK

1. Birth of MCP (2023-2024)
   - Antropic introduces Model Context Protocol (MCP)
     Structured communication b/w LLM & external tools / data.
   - Early needs:
     - build MCP server (tools / resources / prompts)
     - build MCP clients (apps consuming those servers)
   - Official python SDK (mcp) released
     - mcp.server → build servers
     - mcp.client → build clients
     - mcp.cli → CLI for debugging / testing
   - This becomes the reference implementation

2. Develop friction & FastMCP 1.x (2024)
   - Problem: Raw SDK = verbose & boilerplate heavy
     - manual List_tools, call_tool, list_prompts, etc.
     - Explict schema definitions required
   - Solution: FastMCP helper inside SDK (mcp.server.fastMCP)
   - Decorator-driven, type hint aware.

```
from mcp.server.fastmcp import FastMCP
mcp = FastMCP('Demo')
@mcp.tool()
def add(a:int, b:int) → int:
    return a+b
```

3. ~~MCP~~ Fast MCP Break Out (2025)
   - FastMCP adoption grow rapidly
   - SDK maintain's goles:
     - Keep SDK minimal & spec foucsed
     - Let FastMCP evolve faster (CQol APIs, transports, integrations)

Scanned with OKEN Scanner

- FastMCP 2.x released as standalone packages.

4. Where Things Stand (Late 2025)

- MCP SDK
  - official. spec-compliant. stable
  - install: pip install mcp[cli]

- MCP CLI (mcp--)
  - ships with SDK
  - Inspect debug. test servers
- FastMCP 2.x (fast. mcp)
  - standalone, community-driven. fast-evolving
  - Ideal for students educators. rapid prototypes
  - Not offical reference. but more ergonomic

Scanned with OKEN Scanner